In [2]:
```
# install required packages
%pip install -q -U "transformers>=4.50.0" accelerate huggingface_hub safetensors sentencepiece
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/12.8 MB ? eta -:--:--
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.7/12.8 MB 21.5 MB/s eta 0:00:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.4/12.8 MB 93.5 MB/s eta 0:00:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 254.5 MB/s eta 0:00:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 139.9 MB/s eta 0:00:00
```

In [3]:
```
# Deep Learning Functionality
import torch
from transformers import AutoProcessor, AutoModelForImageTextToText

# Image Processing
from PIL import Image

# Data Handling And Analysis
import pandas as pd

# Utilities
import gc
import hashlib
import json
import random
import shutil
import tarfile
import time
from pathlib import Path

# Hugging Face Authentication
from huggingface_hub import login

# Progress Monitoring
from tqdm.auto import tqdm
```

In [4]:
```
# display complete DataFrame columns
pd.set_option("display.max_colwidth", None)
```

In [5]:
```
# authenticate with Hugging Face
try:
```

```
    from google.colab import userdata
    hf_token = userdata.get("HF_TOKEN")
except Exception:
    hf_token = None

if hf_token:
    login(token=hf_token, add_to_git_credential=False)
else:
    login(add_to_git_credential=False)
```



To log in, open this URL and enter the code:

https://hf.co/oauth/device

T J 2 J - A J X S

Waiting for authorization...

Login successful. Logged in as **hodsong** (token: `oauth-hodsong`).

This token will be refreshed automatically when it expires.

In [6]:
```
# project folder
project_folder = Path("/content/drive/MyDrive/OCOM5300M_VLM_Benchmark")

# test captions filename
test_captions_filename = project_folder / "test_captions.csv"

# stage 3 pipeline results
pipeline_test_filename = project_folder / "pipeline_test.csv"

# test image folder and archive
test_image_folder = project_folder / "test_images"
test_image_archive = project_folder / "rocov2_test_images.tar"

# select archive or extracted test images
use_image_archive = False
```

```
# output folder
output_folder = project_folder / "outputs"
output_folder.mkdir(parents=True, exist_ok=True)
```

In [7]:
```
# pipeline-enhanced captions filename
vlm_generated_captions_filename = output_folder / "VLM_pipeline_generated_captions.csv"

# separate checkpoint and diagnostics filenames
checkpoint_filename = output_folder / "VLM_pipeline_generated_captions_checkpoint.csv"
diagnostics_filename = output_folder / "VLM_pipeline_generation_diagnostics.csv"
errors_filename = output_folder / "VLM_pipeline_generation_errors.csv"
configuration_filename = output_folder / "VLM_pipeline_generation_configuration.json"
```

In [8]:
```
# set random seed for reproducibility
seed = 42

random.seed(seed)
torch.manual_seed(seed)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

In [9]:
```
# select GPU
if not torch.cuda.is_available():
    raise RuntimeError("Enable a GPU in Colab: Runtime > Change runtime type > GPU")

device = torch.device("cuda:0")
gpu_name = torch.cuda.get_device_name(0)
gpu_memory_gb = torch.cuda.get_device_properties(0).total_memory / (1024 ** 3)

# select model precision
model_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16

print(f"GPU: {gpu_name}")
print(f"GPU memory: {gpu_memory_gb:.2f} GB")
print(f"Model precision: {model_dtype}")
```

```
GPU: NVIDIA A100-SXM4-40GB
GPU memory: 39.49 GB
Model precision: torch.bfloat16
```

In [10]:
```python
# verify Google Drive files
from pathlib import Path

project_folder = Path(
    "/content/drive/MyDrive/OCOM5300M_VLM_Benchmark"
)

print("Drive mounted:", Path("/content/drive/MyDrive").is_dir())
print("Project folder:", project_folder.is_dir())

print("Test captions:", (project_folder / "test_captions.csv").is_file())
print("Pipeline test:", (project_folder / "pipeline_test.csv").is_file())
print("Test images:", (project_folder / "test_images/test").is_dir())
```

```
Drive mounted: True
Project folder: True
Test captions: True
Pipeline test: True
Test images: True
```

In [11]:
```python
# list project folder contents
from pathlib import Path

project_folder = Path(
    "/content/drive/MyDrive/OCOM5300M_VLM_Benchmark"
)

for item in sorted(project_folder.iterdir()):
    print(item.name, "- folder" if item.is_dir() else "- file")
```

```
outputs - folder
pipeline_test.csv - file
test_captions.csv - file
test_images - folder
train - folder
train_captions.csv - file
train_images - folder
train_images.zip - file
transformer - folder
valid_captions.csv - file
valid_images - folder
valid_images.zip - file
```

In [12]:

```python
# confirm input files exist
if not test_captions_filename.is_file():
    raise FileNotFoundError(f"Test captions file not found: {test_captions_filename}")

if not pipeline_test_filename.is_file():
    raise FileNotFoundError(f"Pipeline test CSV not found: {pipeline_test_filename}")

if use_image_archive and not test_image_archive.is_file():
    raise FileNotFoundError(f"Test image archive not found: {test_image_archive}")

if not use_image_archive and not test_image_folder.is_dir():
    raise FileNotFoundError(f"Test image folder not found: {test_image_folder}")
```

In [13]:

```python
# extract test images if using the archive
if use_image_archive:
    local_folder = Path("/content/roco_medgemma")
    image_folder = local_folder / "test_images"
    extraction_marker = local_folder / "extraction_complete.txt"
    local_folder.mkdir(parents=True, exist_ok=True)

    if not extraction_marker.exists():
        if image_folder.exists():
            shutil.rmtree(image_folder)
        image_folder.mkdir(parents=True, exist_ok=True)

        local_archive = local_folder / test_image_archive.name
        shutil.copy2(test_image_archive, local_archive)
```

```
        # extract the archive safely
        with tarfile.open(local_archive, "r:*") as archive:
            archive.extractall(image_folder, filter="data")

        extraction_marker.write_text("complete", encoding="utf-8")
        local_archive.unlink(missing_ok=True)
else:
    image_folder = test_image_folder

print(f"Test image folder: {image_folder}")
```

```
Test image folder: /content/drive/MyDrive/OCOM5300M_VLM_Benchmark/test_images
```

In [14]:

```
# load test image IDs without using reference captions
test_caption_data = pd.read_csv(test_captions_filename, dtype={"ID": str})

if "ID" not in test_caption_data.columns:
    raise KeyError("The test captions CSV must contain an ID column")

if test_caption_data["ID"].isna().any():
    raise ValueError("The test captions CSV contains missing image IDs")

test_image_ids = test_caption_data["ID"].str.strip().drop_duplicates().tolist()

if "" in test_image_ids:
    raise ValueError("The test captions CSV contains blank image IDs")

print(f"Unique ROCOv2 test images: {len(test_image_ids)}")
```

```
Unique ROCOv2 test images: 9927
```

In [15]:

```
# index test image files
image_extensions = {".jpg", ".jpeg", ".png"}
image_path_lookup = {}

for image_path in image_folder.rglob("*"):
    if image_path.is_file() and image_path.suffix.lower() in image_extensions:
        if image_path.stem in image_path_lookup:
            raise ValueError(f"Duplicate image filename ID: {image_path.stem}")
        image_path_lookup[image_path.stem] = image_path

# confirm test image coverage
```

```
missing_image_ids = [image_id for image_id in test_image_ids if image_id not in image_path_lookup]

print(f"Images available: {len(test_image_ids) - len(missing_image_ids)}")
print(f"Missing images: {len(missing_image_ids)}")

if missing_image_ids:
    raise FileNotFoundError(f"Missing test images, first 10 IDs: {missing_image_ids[:10]}")
```

```
Images available: 9927
Missing images: 0
```

In [16]:

```
# select accepted pipeline concepts
# change these settings before generating captions, not during a resumed run
max_concepts_per_image = 20
include_concept_probabilities = False

# read pipeline CSV without changing CUI identifiers
pipeline_data = pd.read_csv(pipeline_test_filename, dtype=str, keep_default_na=False)

# match CSV headers even when punctuation or spaces differ
normalized_columns = {
    "".join(character for character in column.lower() if character.isalnum()): column
    for column in pipeline_data.columns
}

image_id_column = normalized_columns.get("imageid", normalized_columns.get("id"))
cui_column = normalized_columns.get("cui")
concept_column = normalized_columns.get("conceptname")
reliable_column = normalized_columns.get("reliable")
probability_column = next(
    (column for key, column in normalized_columns.items() if key.startswith("probability")),
    None,
)

if any(column is None for column in [image_id_column, cui_column, concept_column, reliable_column]):
    raise KeyError(
        "Pipeline CSV requires Image ID, CUI, Concept Name and Reliable columns. "
        f"Found: {list(pipeline_data.columns)}"
    )

pipeline_data = pipeline_data.rename(columns={
    image_id_column: "image_id",
```

```
    cui_column: "cui",
    concept_column: "concept_name",
    reliable_column: "reliable",
})

for column in ["image_id", "cui", "concept_name", "reliable"]:
    pipeline_data[column] = pipeline_data[column].astype(str).str.strip()

if probability_column is not None:
    pipeline_data["probability"] = pd.to_numeric(pipeline_data[probability_column], errors="coerce")
else:
    pipeline_data["probability"] = float("nan")

# validate reliable flags
reliable_values = pipeline_data["reliable"].str.lower()
accepted_flags = {"true", "false", "1", "0", "yes", "no", ""}
invalid_flags = set(reliable_values.unique()) - accepted_flags
if invalid_flags:
    raise ValueError(f"Unexpected Reliable values in pipeline CSV: {sorted(invalid_flags)}")

pipeline_data["reliable"] = reliable_values.isin(["true", "1", "yes"])

# prevent an accidental dataset mismatch
unexpected_ids = set(pipeline_data["image_id"]) - set(test_image_ids)
unexpected_ids.discard("")
if unexpected_ids:
    raise ValueError(f"Pipeline test CSV contains IDs outside the test split: {sorted(unexpected_ids)[:10]}")

# build a reliable-concept lookup for each test image
accepted_pipeline_data = (
    pipeline_data.loc[
        pipeline_data["reliable"]
        & pipeline_data["image_id"].ne("")
        & pipeline_data["cui"].ne("")
        & pipeline_data["concept_name"].ne(""),
        ["image_id", "cui", "concept_name", "probability"],
    ]
    .sort_values("probability", ascending=False, na_position="last")
    .drop_duplicates(subset=["image_id", "cui"], keep="first")
)

# group by image ID, keeping highest-probability CUIs first
```

```
pipeline_concepts_by_id = {}
for image_id, group in accepted_pipeline_data.groupby("image_id", sort=False):
    rows = list(group.itertuples(index=False, name=None))
    pipeline_concepts_by_id[image_id] = [
        {
            "cui": cui,
            "name": " ".join(concept_name.split()),
            "probability": probability,
        }
        for _, cui, concept_name, probability in rows[:max_concepts_per_image]
    ]

# fingerprint pipeline inputs to prevent resuming with changed concepts
pipeline_hash = hashlib.sha256()
with pipeline_test_filename.open("rb") as pipeline_file:
    for block in iter(lambda: pipeline_file.read(1024 * 1024), b""):
        pipeline_hash.update(block)
pipeline_sha256 = pipeline_hash.hexdigest()

print(f"Pipeline CSV rows: {len(pipeline_data)}")
print(f"Accepted unique image/CUI pairs: {len(accepted_pipeline_data)}")
print(f"Test images with accepted concepts: {len(pipeline_concepts_by_id)} / {len(test_image_ids)}")
print(f"Maximum accepted concepts per image in prompt: {max_concepts_per_image}")
print(f"Images with more accepted concepts than the cap: "
      f"{sum(size > max_concepts_per_image for size in accepted_pipeline_data.groupby('image_id').size())}")
```

```
Pipeline CSV rows: 1823618
Accepted unique image/CUI pairs: 674931
Test images with accepted concepts: 9905 / 9927
Maximum accepted concepts per image in prompt: 20
Images with more accepted concepts than the cap: 7887
```

In [17]:
```
# pretrained MedGemma model
model_name = "google/medgemma-1.5-4b-it"
```

In [18]:
```
# baseline image-captioning instructions
caption_prompt = (
    "Generate one concise radiology image caption describing only the visible image. "
    "Do not invent findings that are not visible. "
    "Return only the caption."
)
```

```
def build_caption_prompt(image_id):
    """Add accepted pipeline CUIs for this image without presenting them as facts."""
    concepts = pipeline_concepts_by_id.get(image_id, [])

    # preserve the baseline prompt when no concepts were accepted
    if not concepts:
        return caption_prompt

    concept_lines = []
    for concept in concepts:
        concept_text = f"- {concept['cui']}: {concept['name']}"
        if include_concept_probabilities and pd.notna(concept["probability"]):
            concept_text += f" (predicted probability: {concept['probability']:.3f})"
        concept_lines.append(concept_text)

    return (
        f"{caption_prompt}\n\n"
        "Additional image-derived candidate concepts accepted by symbolic and "
        "conformal filtering (not confirmed diagnoses):\n"
        + "\n".join(concept_lines)
        + "\nUse these candidates only when supported by the visible image. "
        "They may be incomplete or wrong; do not assert a finding solely because "
        "it appears in this list. Do not include CUI codes in the caption."
    )
```

In [19]:

```
# caption generation settings
maximum_new_tokens = 96

generation_configuration = {
    "max_new_tokens": maximum_new_tokens,
    "do_sample": False,
    "num_beams": 1,
    "use_cache": True,
}

generation_configuration
```

Out[19]:

```
{'max_new_tokens': 96, 'do_sample': False, 'num_beams': 1, 'use_cache': True}
```

In [20]:
```
# Load MedGemma processor
processor = AutoProcessor.from_pretrained(model_name)
```

```
processor_config.json: 100% 70.0/70.0 [00:00<00:00, 8.77kB/s]
chat_template.jinja: 100% 1.53k/1.53k [00:00<00:00, 200kB/s]
preprocessor_config.json: 100% 570/570 [00:00<00:00, 76.2kB/s]
config.json: 100% 2.55k/2.55k [00:00<00:00, 324kB/s]
tokenizer_config.json: 100% 1.16M/1.16M [00:00<00:00, 34.2MB/s]
tokenizer.json: reconstructing file: 100% 33.4MB / 33.4MB, 3.29MB/s
tokenizer.json: downloading bytes: 8.82MB, 868kB/s
added_tokens.json: 100% 35.0/35.0 [00:00<00:00, 4.77kB/s]
special_tokens_map.json: 100% 662/662 [00:00<00:00, 94.6kB/s]
```

In [21]:
```
# Load pretrained MedGemma model
medgemma = AutoModelForImageTextToText.from_pretrained(
    model_name,
    torch_dtype=model_dtype,
    device_map="auto",
    low_cpu_mem_usage=True,
)

# set evaluation mode
medgemma.eval()
model_device = next(medgemma.parameters()).device

print(f"Model loaded: {model_name}")
print(f"Model device: {model_device}")
```

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors.index.json: 100% 90.6k/90.6k [00:00<00:00, 9.48MB/s]
Download complete: 7.79GB, 1.16GB/s
```

Reconstruction complete: 100% 8.60GB / 8.60GB, 466MB/s

Fetching 2 files: 100% 2/2 [00:25<00:00, 25.65s/it]

Loading weights: 100% 883/883 [00:02<00:00, 1031.23it/s]

generation_config.json: 100% 115/115 [00:00<00:00, 15.5kB/s]

```
Model loaded: google/medgemma-1.5-4b-it
Model device: cuda:0
```

In [22]:

```python
def clean_generated_caption(caption):
    """
    Remove additional whitespace and an optional caption prefix.

    Args:
        caption (str): decoded MedGemma response

    Returns:
        str: cleaned generated caption
    """
    try:
        caption = " ".join(str(caption).strip().split())

        if caption.lower().startswith("caption:"):
            caption = caption[len("caption:"):].strip()

        return caption

    except Exception as e:
        raise RuntimeError(f"Exception: {e}")
```

In [23]:

```python
def generate_medgemma_caption(image_id, image_path):
    """
    Generate one caption for a ROCOv2 test image.

    Args:
        image_id (str): ROCOv2 image ID
        image_path (Path): path to the test image

    Returns:
        tuple: generated caption and inference diagnostics
```

```
    """
    try:
        # load RGB image
        with Image.open(image_path) as image_file:
            image = image_file.convert("RGB")

        # construct image-specific prompt from reliable CUIs
        image_caption_prompt = build_caption_prompt(image_id)
        messages = [{
            "role": "user",
            "content": [
                {"type": "image", "image": image},
                {"type": "text", "text": image_caption_prompt},
            ],
        }]

        # prepare model inputs
        inputs = processor.apply_chat_template(
            messages,
            add_generation_prompt=True,
            tokenize=True,
            return_dict=True,
            return_tensors="pt",
        ).to(model_device, dtype=model_dtype)

        input_token_count = inputs["input_ids"].shape[-1]

        # generate caption
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()
        start_time = time.perf_counter()

        with torch.inference_mode():
            output_tokens = medgemma.generate(**inputs, **generation_configuration)

        torch.cuda.synchronize()
        generation_seconds = time.perf_counter() - start_time

        # decode generated tokens
        generated_tokens = output_tokens[0, input_token_count:]
        generated_caption = clean_generated_caption(
            processor.decode(generated_tokens, skip_special_tokens=True)
```

```
        )

        if not generated_caption:
            raise ValueError("MedGemma generated an empty caption")

        # record generation diagnostics
        diagnostics = {
            "Input Tokens": int(input_token_count),
            "Output Tokens": int(generated_tokens.shape[0]),
            "Generated Words": len(generated_caption.split()),
            "Generation Seconds": generation_seconds,
            "CUDA Peak Allocated GB": torch.cuda.max_memory_allocated() / (1024 ** 3),
        }

        del inputs, output_tokens, generated_tokens, image
        return generated_caption, diagnostics

    except Exception as e:
        raise RuntimeError(f"Exception: {e}")
```

In [24]:

```
# save model and prompt configuration
run_configuration = {
    "model_name": model_name,
    "caption_prompt": caption_prompt,
    "pipeline_file": pipeline_test_filename.name,
    "pipeline_sha256": pipeline_sha256,
    "pipeline_filter": "Reliable TRUE",
    "max_concepts_per_image": max_concepts_per_image,
    "include_concept_probabilities": include_concept_probabilities,
    "generation_configuration": generation_configuration,
    "seed": seed,
    "test_image_count": len(test_image_ids),
}

# verify previous run configuration
previous_predictions_exist = checkpoint_filename.exists() or vlm_generated_captions_filename.exists()

if previous_predictions_exist and configuration_filename.exists():
    previous_configuration = json.loads(configuration_filename.read_text(encoding="utf-8"))
    if previous_configuration != run_configuration:
        raise ValueError("Existing predictions use different model or prompt settings. Start with a new output
```

```
folder.")
elif previous_predictions_exist:
    raise ValueError("Existing predictions have no configuration file. Start with a new output folder.")

configuration_filename.write_text(json.dumps(run_configuration, indent=4), encoding="utf-8")
```

Out[24]:
```
670
```

In [25]:
```
# load previously generated captions
resume_filename = checkpoint_filename if checkpoint_filename.exists() else vlm_generated_captions_filename

if resume_filename.exists():
    previous_captions = pd.read_csv(resume_filename, dtype={"ID": str})
    if previous_captions["ID"].duplicated().any():
        raise ValueError("Saved predictions contain duplicate image IDs")
    previous_captions = previous_captions.dropna(subset=["ID", "generated_caption"])
    caption_records = dict(zip(previous_captions["ID"], previous_captions["generated_caption"]))
else:
    caption_records = {}

# load previous diagnostics and errors
if diagnostics_filename.exists():
    generation_diagnostics = pd.read_csv(diagnostics_filename, dtype={"ID": str}).set_index("ID").to_dict("index")
else:
    generation_diagnostics = {}

if errors_filename.exists():
    previous_errors = pd.read_csv(errors_filename, dtype={"ID": str})
    generation_errors = dict(zip(previous_errors["ID"], previous_errors["Error"]))
else:
    generation_errors = {}

# identify remaining test images
remaining_image_ids = [image_id for image_id in test_image_ids if image_id not in caption_records]

print(f"Captions already generated: {len(caption_records)}")
print(f"Images remaining: {len(remaining_image_ids)}")
```

```
Captions already generated: 0
Images remaining: 9927
```

In [26]:
```
def save_generation_progress():
    """
    Save the current captions, diagnostics and errors to Google Drive.

    Returns:
        None
    """
    # retain original test image order
    caption_data = pd.DataFrame(
        [(image_id, caption_records[image_id]) for image_id in test_image_ids if image_id in caption_records],
        columns=["ID", "generated_caption"],
    )
    caption_data.to_csv(checkpoint_filename, index=False)

    diagnostic_data = pd.DataFrame(
        [{"ID": image_id, **values} for image_id, values in generation_diagnostics.items()],
        columns=["ID", "Input Tokens", "Output Tokens", "Generated Words", "Generation Seconds", "CUDA Peak
Allocated GB"],
    )
    diagnostic_data.to_csv(diagnostics_filename, index=False)

    error_data = pd.DataFrame(
        [{"ID": image_id, "Error": error} for image_id, error in generation_errors.items()],
        columns=["ID", "Error"],
    )
    error_data.to_csv(errors_filename, index=False)
```

In [27]:
```
# generate one pipeline-enhanced test caption
run_single_image_test = True

if run_single_image_test:
    # choose an unprocessed image with accepted concepts when possible
    candidates = remaining_image_ids if remaining_image_ids else test_image_ids
    single_image_id = next(
        (image_id for image_id in candidates if pipeline_concepts_by_id.get(image_id)),
        candidates[0],
    )
    single_image_path = image_path_lookup[single_image_id]

    print(f"Image ID: {single_image_id}")
    print("Prompt used:")
```

```
    print(build_caption_prompt(single_image_id))

    single_caption, single_diagnostics = generate_medgemma_caption(single_image_id, single_image_path)

    print(f"Generated Caption: {single_caption}")
    print(single_diagnostics)
```

```
Image ID: ROCOv2_2023_test_000001
Prompt used:
Generate one concise radiology image caption describing only the visible image. Do not invent findings that are not v
isible. Return only the caption.

Additional image-derived candidate concepts accepted by symbolic and conformal filtering (not confirmed diagnoses):
- C0040405: X-Ray Computed Tomography
- C0034065: Pulmonary Embolism
- C0456973: Hilar lymphadenopathy
- C0025066: Mediastinum
- C0520743: Mediastinal lymphadenopathy
- C0230148: Anterior mediastinum
- C0205076: Chest wall structure
- C0019123: Hemothorax
- C1305372: Hilar lymph node
- C0225630: Left main bronchus structure
- C0230150: Posterior mediastinum
- C0226054: Right pulmonary artery
- C0225608: Right main bronchus structure
- C0014876: Esophagus
- C0178802: Pleural cavity
- C0034052: Pulmonary artery structure
- C0226069: Left pulmonary artery
- C0013990: Pathological accumulation of air in tissues
- C0031050: Pericardial sac structure
- C0032225: Pleura
Use these candidates only when supported by the visible image. They may be incomplete or wrong; do not assert a findi
ng solely because it appears in this list. Do not include CUI codes in the caption.
Generated Caption: Chest X-ray showing the heart, lungs, and mediastinum.
{'Input Tokens': 653, 'Output Tokens': 15, 'Generated Words': 8, 'Generation Seconds': 3.5425530070000377, 'CUDA Peak
Allocated GB': 8.177570343017578}
```

In [28]:
```
# enable full test set generation when ready
run_full_test_set = False
```

```
# checkpoint frequency
checkpoint_every = 25
```

In [29]:
```
run_full_test_set = True
```

In [30]:
```
# generate remaining captions
if run_full_test_set:
    newly_generated = 0

    try:
        for image_id in tqdm(remaining_image_ids, desc="MedGemma captions", unit="image"):
            generation_succeeded = False

            try:
                generated_caption, diagnostics = generate_medgemma_caption(image_id, image_path_lookup[image_id])

                caption_records[image_id] = generated_caption
                generation_diagnostics[image_id] = diagnostics
                generation_errors.pop(image_id, None)
                newly_generated += 1
                generation_succeeded = True

            except Exception as e:
                generation_errors[image_id] = str(e)

            # save generation progress
            if generation_succeeded and newly_generated % checkpoint_every == 0:
                save_generation_progress()

                gc.collect()
                torch.cuda.empty_cache()

    finally:
        # save completed predictions
        save_generation_progress()

    print(f"Generated captions saved in checkpoint: {len(caption_records)}")
    print(f"Images with recorded errors: {len(generation_errors)}")
```

```
MedGemma captions: 100% 9927/9927 [5:22:38<00:00, 2.91s/image]
```

```
Generated captions saved in checkpoint: 9927
Images with recorded errors: 0
```

In [31]:
```python
# construct generated captions DataFrame
vlm_generated_captions = pd.DataFrame(
    [(image_id, caption_records[image_id]) for image_id in test_image_ids if image_id in caption_records],
    columns=["ID", "generated_caption"],
)

# confirm prediction coverage
expected_image_count = len(test_image_ids)
generated_image_count = len(vlm_generated_captions)

print(f"Expected ROCOv2 test images: {expected_image_count}")
print(f"Generated captions: {generated_image_count}")
print(f"Remaining images: {expected_image_count - generated_image_count}")

vlm_generated_captions.head()
```

```
Expected ROCOv2 test images: 9927
Generated captions: 9927
Remaining images: 0
```

Out[31]:

	ID	generated_caption
0	ROCOv2_2023_test_000001	Chest X-ray showing the heart, lungs, and mediastinum.
1	ROCOv2_2023_test_000002	Coronary artery, left.
2	ROCOv2_2023_test_000003	Angiogram of the left common carotid artery, showing a stent in place.
3	ROCOv2_2023_test_000004	Angiogram of the left internal thoracic artery with a stent in place.
4	ROCOv2_2023_test_000005	Radiograph of the right shoulder, showing the glenohumeral joint. The image displays the humeral head and glenoid fossa. The angle between the humeral head and glenoid fossa is approximately 128.6 degrees. The angle between the humeral head and the glenoid fossa is approximately 73.3 degrees.

In [32]:
```python
# validate complete test set
valid_captions = (
    vlm_generated_captions["generated_caption"].notna().all()
    and vlm_generated_captions["generated_caption"].astype(str).str.strip().ne("").all()
)
```

```
complete_test_set = (
    generated_image_count == expected_image_count
    and vlm_generated_captions["ID"].is_unique
    and set(vlm_generated_captions["ID"]) == set(test_image_ids)
    and valid_captions
)

if complete_test_set:
    # save CSV for Performance Notebook
    vlm_generated_captions = vlm_generated_captions[["ID", "generated_caption"]]
    vlm_generated_captions.to_csv(vlm_generated_captions_filename, index=False)
    print(f"Final CSV saved: {vlm_generated_captions_filename}")
else:
    print("The full test set is not complete. Resume generation before exporting the final CSV.")
    print(f"Checkpoint: {checkpoint_filename}")
```

```
Final CSV saved: /content/drive/MyDrive/OCOM5300M_VLM_Benchmark/outputs/VLM_pipeline_generated_captions.csv
```

In [33]:
```
# display generated caption statistics
if not vlm_generated_captions.empty:
    generated_caption_lengths = vlm_generated_captions["generated_caption"].astype(str).str.split().str.len()
    print(generated_caption_lengths.describe())
    print(f"Unique generated captions: {vlm_generated_captions['generated_caption'].nunique()}")
else:
    print("No captions generated yet")
```

```
count    9927.000000
mean       19.551526
std        19.112158
min         2.000000
25%         9.000000
50%        12.000000
75%        22.000000
max        90.000000
Name: generated_caption, dtype: float64
Unique generated captions: 7576
```


Google (2026) *MedGemma 1.5 4B model documentation*. Available At https://developers.google.com/health-ai-developer-foundations/medgemma/model-card

Hugging Face (2026) *google/medgemma-1.5-4b-it*. Available At https://huggingface.co/google/medgemma-1.5-4b-it

Hugging Face (2026) *Transformers Documentation*. Available At https://huggingface.co/docs/transformers/

Google (2026) *Google Colaboratory*. Available At https://colab.research.google.com/